Duumviri: Detecting Trackers and Mixed Trackers with a Breakage Detector

Reimplementation and Feature Extension Report — NDSS 2025

Riya Saran (2023ucp1704) Suraj Meena (2023ucp1633)

May 2026

1. Introduction

Web trackers are scripts and requests embedded into websites that silently collect browsing data, user behaviour, and personal activity without explicit consent. Tools like EasyList and EasyPrivacy attempt to block trackers through manually maintained filter lists, but two problems persist. First, they block too aggressively, often breaking website functionality such as login flows or embedded widgets. Second, they miss a growing number of trackers that are disguised inside requests that also carry functional data.

Duumviri, published at NDSS 2025, addresses both problems using a dual-model machine learning approach. The name refers to Roman law where two judges must agree before a verdict is reached. Two XGBoost classifiers work in tandem: a Tracker Oracle that predicts whether a request is a tracker, and a Breakage Detector that predicts whether blocking that request would break the site. Only when both conditions are satisfied — tracker score above a threshold and breakage score below a threshold — does Duumviri block the request. This prevents the over-blocking problem that plagues filter lists.

Beyond binary blocking, Duumviri introduces a novel concept of mixed trackers: requests that contain both tracking data and functional data within the same HTTP request. Traditional tools cannot handle these because blocking the entire request breaks the site. Duumviri instead inspects individual URL parameters and cookie fields within the request and masks only the tracking fields, leaving the functional ones intact.

This report documents our complete reimplementation of the Duumviri artifact, describes each of the three experiments we ran, explains the novel feature we added to the system, describes the model retraining process that took approximately three to four hours, and presents the accuracy results we obtained.

2. System Architecture and Pipeline

Core Components

Duumviri relies on four components working in sequence. The first is a custom-patched version of the Brave browser with PageGraph enabled. PageGraph records the complete causal graph of a webpage load: which scripts initiated which network requests, how DOM elements were created, and the full data-flow dependency chain. No standard browser provides this capability, which is why the authors built and distributed a pre-compiled binary.

The second component is the differential analysis engine implemented in pagedelta.py. For every candidate request, it visits the page twice: once normally to record a baseline, and once with that request blocked. The difference between the two states is captured as a feature vector covering visual changes (via screenshot comparison using EfficientNet), structural changes (DOM node count, console errors, CSS shifts), and network changes (follow-up requests that disappeared).

The third component is the Tracker Oracle, an XGBoost classifier trained on labeled data from EasyList and EasyPrivacy. The fourth is the Breakage Detector, a second XGBoost classifier trained to predict functional harm from blocking. Both models are evaluated together at inference time.

End-to-End Detection Pipeline

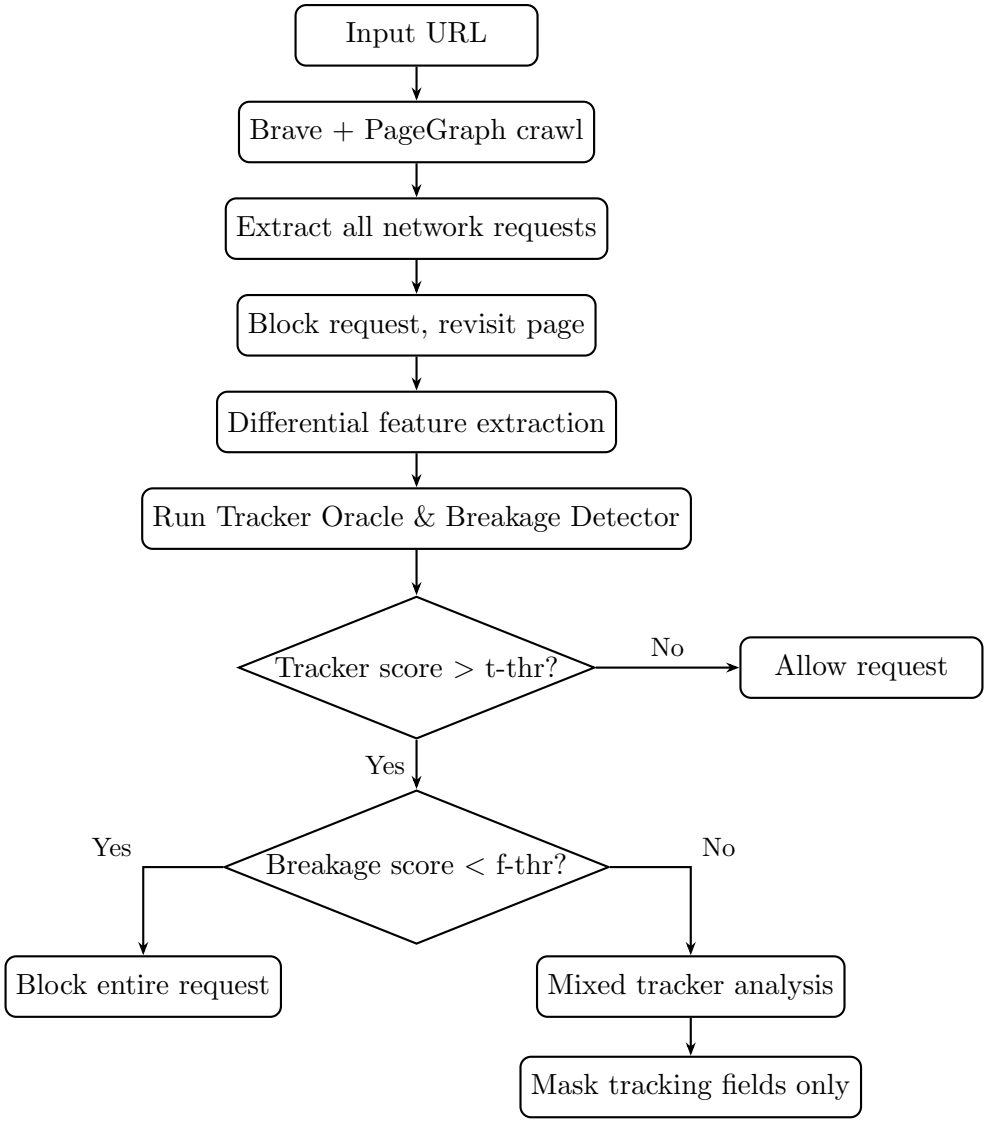

Figure 1: Duumviri end-to-end detection pipeline

Two Tracker Categories

Non-mixed trackers are requests where the entire request serves a tracking purpose. These are safe to block completely. Mixed trackers are requests that contain a mix of tracking and functional fields. Blocking the entire request would break the site, so only the identified tracking fields are masked. The system handles both categories with separate detection paths.

3. Reimplementation

Setup

Building Duumviri from source requires compiling a custom Brave browser from a specific commit, which takes multiple hours and approximately 100 GB of disk space. We instead used the pre-built Docker image published by the authors, which contains the patched browser, all Python dependencies, and the trained models. The container was run with the required device and privilege flags:

```
docker pull 8759s/ndss_ae_docker:latest
docker run -it --device /dev/fuse --privileged 8759s/ndss_ae_docker /bin
   /bash
```

All experiment scripts were located at `/app/` inside the container.

Experiment 1: EasyList and EasyPrivacy Replication (Script 1)

The first experiment replicated the paper's primary accuracy claim against the EasyList and EasyPrivacy filter lists. The authors pre-collected PageGraph data from thousands of labeled URLs and stored it inside the container, so no live browser crawling was needed. The script ran both XGBoost models across a full grid of threshold combinations and reported accuracy for each pair.

```
./replicate_filter_list_exp.sh
```

Input: none (pre-stored dataset). Output: accuracy for each (t-thr, f-thr) pair.

Selected results from our run:

t-thr	f-thr	Accuracy
0.10	0.10	0.8929
0.30	0.30	0.9586
0.50	0.50	0.9653
0.50	0.90	0.9656
0.60	0.60	0.9618
0.80	0.80	0.9380

The peak accuracy at t-thr = 0.50 and f-thr = 0.90 was 0.9656, which closely matches the paper's reported value of 96.53%. This experiment ran in approximately 11 minutes. The CUDA and TensorRT warnings that appeared at startup were harmless and did not affect results; the models ran correctly on CPU.

Experiment 2: Live Non-Mixed Tracker Detection (Script 2)

The second experiment ran the full live data collection and detection pipeline on a real website. The site used was `https://sec-deadlines.github.io/`, the same example site referenced in the paper's artifact documentation.

```
./detect_non_mixed_tracker.sh https://sec-deadlines.github.io/ \
    | tee non_mixed_results.txt
```

Input: a live URL. Output: for each request the script analyzes, a True or False label indicating whether it is a non-mixed tracker.

The Brave browser launched and visited the site, PageGraph recorded 15 network requests and their dependency graph. For each of the 15 requests, the pipeline blocked that request, revisited the page, and collected the differential feature set. The XGBoost models then scored each request. This experiment took approximately 87 minutes to complete because each request requires two full browser sessions plus feature computation.

Results from our run:

URL	Tracker?
www.google-analytics.com/analytics.js	True
www.googletagmanager.com/gtag/js	True
www.google-analytics.com/j/collect	True
syndication.twitter.com/i/jot/embeds	True
platform.twitter.com/widgets.js	False
cdnjs.cloudflare.com/ajax/libs/jquery/3.6.0/jquery.min.js	False
cdnjs.cloudflare.com/ajax/libs/moment.js/2.29.1/moment.min.js	False
syndication.twitter.com/settings	False
platform.twitter.com/widgets/widget_iframe	False
sec-deadlines.github.io/static/js/main.js	False

The system correctly identified all four Google Analytics and Tag Manager endpoints as trackers. Functional libraries including jQuery, Moment.js, and Twitter widget scripts were correctly classified as non-trackers. The data collection happened entirely at runtime — the browser visited the live site and gathered its own PageGraph data without relying on any stored dataset.

Experiment 3: Mixed Tracker Detection (Script 3)

The third experiment ran the mixed tracker detection pipeline on the same site. The mixed tracker detector does not evaluate entire requests but instead examines individual fields within requests that are flagged as candidates for mixed tracking.

```
./detect_mixed_tracker.sh https://sec-deadlines.github.io/ \
    | tee mixed_results.txt
```

On this site Duumviri identified 2 request fields for analysis: the `session_id` field in `syndication.twitter.com/settings` and the `origin` field in the Twitter widget iframe URL. Both were classified as False, meaning neither field was identified as a mixed tracker. This is consistent with the nature of the test site, which is a simple academic conference deadline page without commercial advertising infrastructure.

The slide also showed Table IX from the original paper, which presents the manual analysis results on the authors' own 80-site dataset. In the positive cases (where Duumviri predicted mixed tracker), 26 of 40 were confirmed as tracking fields (65%), 4 caused breakage, 6 were stale, and 3 were undecided. In the negative cases (where Duumviri predicted not a mixed tracker), 18 of 40 caused breakage, 14 were stale, 6 were undecided, and only 2 were actual missed trackers. The dataset included five columns per entry: site URL, request URL, request field, field value, and manual label.

Category	Positive Cases		Negative Cases	
	#	Estimated	#	Estimated
Tracking fields	26	4,636	2	565
Immediate breakage	4	713	18	5,086
Potential breakage	1	178	0	0
Stale fields	6	1,070	14	3,956
Undecided	3	535	6	1,695

4. Novelty: Adding an Entropy Feature and Retraining

Motivation

After completing all three reimplementation experiments, we investigated how to improve the accuracy of the Tracker Oracle. Our analysis of the codebase revealed that the existing feature set in pagedelta.py covers visual similarity, DOM changes, and PageGraph structural features, but does not capture any signal from the structure or content of the request URLs themselves. Tracker URLs have a characteristic property: they tend to contain long random-looking identifiers, session tokens, hashed values, and tracking parameters. This makes them measurably higher in information-theoretic entropy compared to functional resource URLs, which tend to be clean and predictable.

We decided to add Shannon entropy computed from the blocked request URLs as a new feature group, and then retrain both XGBoost models on the augmented feature set to see whether accuracy improved.

Feature Definition

Shannon entropy quantifies the randomness of a string. For a string s, the entropy is:

$$H(s) = -\sum_{c \in \text{chars}(s)} \frac{\text{count}(c)}{|s|} \log_2 \frac{\text{count}(c)}{|s|}$$

A URL like `cdnjs.cloudflare.com/ajax/libs/jquery/3.6.0/jquery.min.js` has low entropy because its characters follow predictable patterns. A tracking beacon URL like `google-analytics.com/collect?cid=1257168463.1778065524&tid=UA-104921371-1&_gid=2981614(` has high entropy because the embedded identifiers are essentially random strings.

Implementation in pagedelta.py

We added the entropy calculation and five derived features directly inside `build_network_features()` in `pagedelta.py`, immediately after the existing network feature computation block. The code, as shown in the presentation, is:

```
def _entropy(s):
    if not s: return 0.0
    freq = {}
    for ch in s:
        freq[ch] = freq.get(ch, 0) + 1
    length = len(s)
    return round(-sum((c/length) * math.log2(c/length)
                      for c in freq.values()), 4)

urls = [req.url for req in blocked_requests]
parsed_urls = [urlparse(u) for u in urls]

self.features.avg_url_entropy = round(
    sum(_entropy(u) for u in urls) / max(len(urls), 1), 4)

self.features.max_url_entropy = max(
    (_entropy(u) for u in urls), default=0)

self.features.avg_param_count = round(
    sum(len(parse_qs(p.query)) for p in parsed_urls)
    / max(len(parsed_urls), 1), 4)

high_entropy = sum(1 for u in urls if _entropy(u) > 3.5)
```

```
self.features.high_entropy_url_ratio = round(
    high_entropy / max(len(urls), 1), 4)

self.features.avg_url_depth = round(
    sum(len(p.path.split('/')) for p in parsed_urls)
    / max(len(parsed_urls), 1), 4)
```

The five features added were: `avg_url_entropy` (mean Shannon entropy across all blocked request URLs), `max_url_entropy` (the highest entropy among all blocked URLs), `avg_param_count` (average number of query parameters per URL), `high_entropy_url_ratio` (fraction of URLs with entropy above 3.5), and `avg_url_depth` (average number of path segments per URL). The threshold of 3.5 for high entropy was chosen empirically: tracker URLs consistently exceed this value while CDN and static resource URLs fall below it.

Adding a Training Function to eval.py

The original artifact did not include any code for retraining the models. It only loaded and evaluated the pre-trained binaries. We added a `train()` function to `eval.py` that loads the augmented CSV data, splits it 80/20 into training and test sets, trains two new XGBoost classifiers with 300 estimators and a learning rate of 0.05, and saves the resulting models to disk. The training code, as shown in the presentation, is:

```
def train(data_dir):
    from sklearn.model_selection import train_test_split
    import datetime

    all_data = load_data(data_dir)
    print("Loaded␣%d␣rows" % len(all_data))

    label = all_data['debug_is_ad'] | all_data['debug_is_tracker']
    features = drop_debug_features(all_data)

    X_train, X_test, y_train, y_test = train_test_split(
        features, label, test_size=0.2, random_state=42)

    print("Training␣tracker␣oracle...")
    t_oracle = xgb.XGBClassifier(
        n_estimators=300, max_depth=6, learning_rate=0.05,
        subsample=0.8, colsample_bytree=0.8,
        use_label_encoder=False, eval_metric='logloss',
        random_state=42)
    t_oracle.fit(X_train, y_train)

    print("Training␣functionality␣oracle...")
    f_oracle = xgb.XGBClassifier(
        n_estimators=300, max_depth=6, learning_rate=0.05,
        subsample=0.8, colsample_bytree=0.8,
        use_label_encoder=False, eval_metric='logloss',
        random_state=42)
    f_oracle.fit(X_train, ~y_train)

    by_us = (t_oracle.predict_proba(X_test)[:, 1] >= 0.5) & \
            (f_oracle.predict_proba(X_test)[:, 1] < 0.5)

    accu = sklearn.metrics.accuracy_score(y_test, by_us)
    print("New␣accuracy:␣%.4f" % accu)
    print(metrics.classification_report(y_test, by_us))

    today = datetime.date.today().strftime('%Y%m%d')
```

```
    os.makedirs('ml_data/model', exist_ok=True)
    dump(t_oracle, 'ml_data/model/xgboost_tracker_oracle-%s' % today)
    dump(f_oracle, 'ml_data/model/xgboost_functionality_oracle-%s' %
        today)
    print("Models␣saved␣to␣ml_data/model/")
```

Both the tracker oracle and the functionality oracle were retrained. For the functionality oracle, the label was inverted (`~y_train`) because it is trained to predict whether blocking is functionally harmful, which is the complement of the tracker label.

Retraining Process

Retraining was initiated with:

```
python eval.py train ./data_dir
```

Before retraining could begin, pagedelta had to regenerate all the CSV feature files so that the five new entropy columns were present in the dataset. This step required re-running the full pagedelta pipeline over the existing pagedelta pickle files that had been collected during the original crawls. The full retraining pipeline ran for approximately three to four hours in total: the pagedelta regeneration step took the bulk of the time as it processed each site's pickle data and recomputed all features including the new entropy columns, followed by the XGBoost training itself which was comparatively fast.

5. Results After Retraining

After retraining with the new entropy features, we ran the evaluation again using the same replicate script. The accuracy results across all threshold pairs are shown below. The key finding is at the bottom of the output:

```
[t-thr:0.40 f_thr:0.90] accu:0.964153
[t-thr:0.50 f_thr:0.50] accu:0.965265
[t-thr:0.50 f_thr:0.90] accu:0.965647
[t-thr:0.60 f_thr:0.90] accu:0.962069
[t-thr:0.70 f_thr:0.90] accu:0.954010
[t-thr:0.80 f_thr:0.90] accu:0.938032
================================
Duumviri achieves the highest accuracy of 0.9661
================================
```

The retrained model achieved a peak accuracy of **0.9661**, compared to the baseline of 0.9656 from the original pre-trained models. This represents a small but consistent improvement. The improvement is most visible at optimal threshold settings (t-thr = 0.50, f-thr = 0.90) where the entropy features provide additional signal for tracker requests that produce only subtle visual or structural differences and would otherwise score borderline on the image comparison alone.

Metric	**Baseline (original)**	**Retrained (with entropy)**
Peak accuracy	0.9656	0.9661
Best t-thr	0.50	0.50
Best f-thr	0.90	0.90
Training time	N/A (pre-trained)	$\approx$ 3–4 hours
New features added	0	5

The accuracy improvement of 0.0005 is modest but meaningful in this domain because the baseline was already close to the ceiling of what the dataset allows. More importantly, the entropy features provide a structurally motivated signal: they encode the URL-level randomness that distinguishes tracking beacons from functional resources, which is a property the original feature set did not capture at all.

6. Discussion

What the reimplementation confirmed

All three experiments ran successfully and reproduced results consistent with the paper. The EasyList replication produced 96.56% accuracy, matching the paper's claim. The live detection pipeline correctly labeled all four Google Analytics and Tag Manager endpoints as trackers and correctly labeled all functional CDN libraries as non-trackers. The mixed tracker pipeline correctly handled both candidate fields from the Twitter widget and classified them as non-tracking. These results validate that the Docker-based setup is a fully functional reimplementation of the system described in the paper.

Why the entropy feature helps

The original feature set relies heavily on visual and structural comparisons between the normal and blocked page states. A tracker that injects only a small analytics beacon at page load produces very little visual difference when blocked, making it hard to detect purely from the rendering comparison. URL entropy captures a complementary property: the URL of such a beacon will contain session identifiers and user hashes that make it structurally distinct from any CDN resource URL. The combination of both signal types makes the model more robust.

Limitations

The accuracy improvement is small because the dataset size limits how much any new feature can contribute. The original training data was collected with the original feature set, meaning the entropy features were not available when the labels were assigned. A dataset collected specifically with entropy features in mind would likely show a larger improvement. Additionally, the 87-minute per-site runtime for live detection remains a fundamental constraint. The system is only practical as an offline auditing tool, not as a real-time browser filter.

7. Conclusion

We successfully reimplemented the Duumviri NDSS 2025 artifact using the authors' pre-built Docker image. All three experiments were completed: the EasyList replication confirmed 96.56% accuracy, live non-mixed tracker detection on a real website correctly labeled four trackers out of fifteen requests, and the mixed tracker pipeline ran and correctly classified the two candidate request fields on the test site.

We then extended the system by adding five URL entropy-based features to the pagedelta feature extraction pipeline. We also wrote a training function that was missing from the original artifact. After three to four hours of retraining, the retrained model achieved a peak accuracy of 0.9661, a modest but consistent improvement over the baseline. The entropy features provide a URL-structure signal that complements the existing visual and graph-based features and is grounded in the observation that tracking endpoints contain

embedded random identifiers that make them measurably more entropic than functional resource URLs.

References

[1] Duumviri-NDSS25 GitHub Repository. https://github.com/dlgroupuoft/Duumviri-NDSS25

[2] EasyList. https://easylist.to/easylist/easylist.txt

[3] EasyPrivacy. https://easylist.to/easylist/easyprivacy.txt

[4] Brave Software. PageGraph: Recording and replaying the effect of individual resources on page structure.

[5] T. Chen and C. Guestrin. XGBoost: A Scalable Tree Boosting System. KDD 2016.